

Московский физико-технический институт
Высшая школа программной инженерии

Формальные языки и сложность вычислений
3 семестр, 2026/2027 уч. год

Лекция 3
Эквивалентность определений NP.
NP-трудность, NP-полнота. coNP.
Сводимость по Карпу

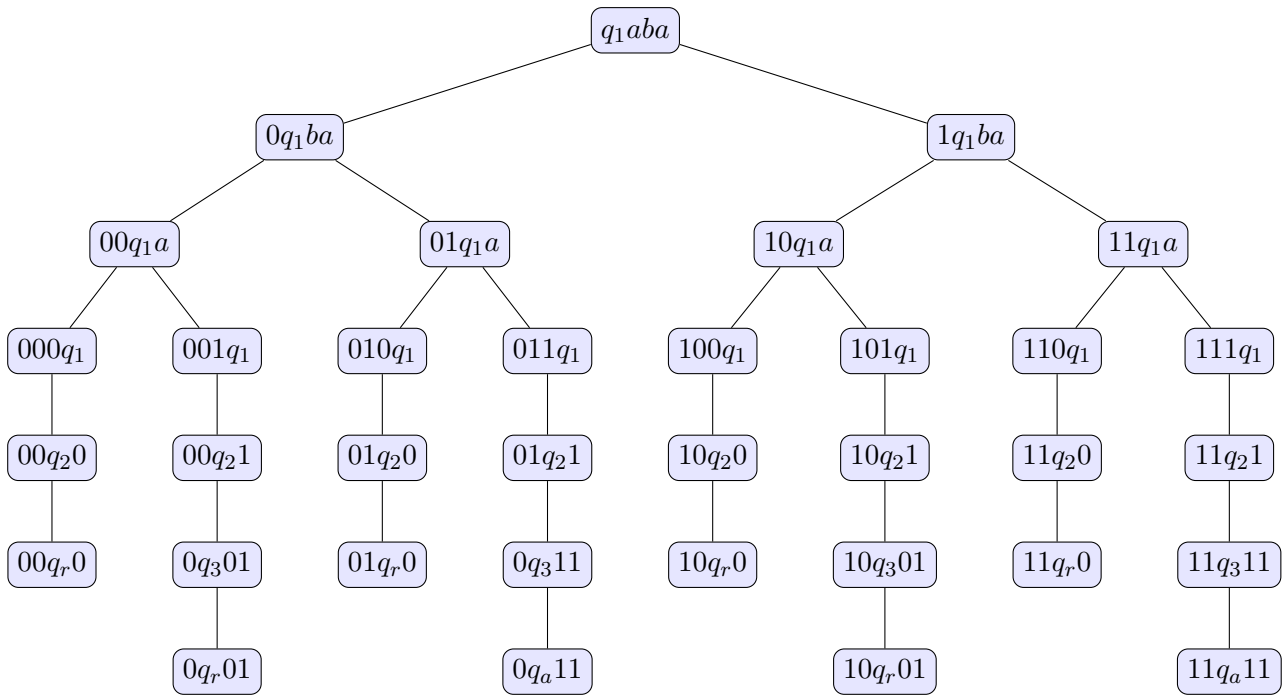
Лектор: Зухба А.В. <3
Автор: [Соля](#)

1 Пример НМТ

Пусть M принимает x , если существует слово в бинарном алфавите длины $|x|$, которое даёт остаток 3 при делении на 4, если рассматривать его как число в двоичной записи. Рассмотрим таблицу переходов, отмечая красным цветом состояния обработки ошибок.

	q_1	q_2	q_3
a, b, c	$(0, +1, q_1), (1, +1, q_1)$	$(0, 0, q_r)$	$(0, 0, q_r)$
0	$(0, 0, q_r)$	$(0, 0, q_r)$	$(0, 0, q_r)$
1	$(1, 0, q_r)$	$(1, -1, q_3)$	$(1, 0, q_a)$
Λ	$(\Lambda, -1, q_2)$	$(\Lambda, 0, q_r)$	$(\Lambda, 0, q_r)$

Изобразим работу МТ на входе $x = aba$, пользуясь конфигурациями:



Каждая ветка дерева по длине будет $\leq |x| + 3$. Вершин в дереве будет $\leq 2^0 + 2^1 + \dots + 2^{|x|} + 3 \cdot 2^{|x|} = 5 \cdot 2^{|x|} - 1$. О чём это нам говорит? Если писать эмуляцию нашей НМТ (которая работает $\leq |x| + 3$) напрямую (рассматривая все возможные достижимые конфигурации), то время работы будет ограничено как $\leq 5 \cdot 2^{|x|} - 1$.

В общем случае посмотрим на нашу таблицу переходов δ . В ней всегда есть какая-то ячейка с наибольшим числом ветвлений. Это число обозначим за m (в нашем примере $m = 2$). Важно заметить, что это характеристика МТ (не зависит от входа), а значит, для нас это будет константа. Тогда на самом деле МТ переберёт последовательно все пути в дереве за $O(m^{T(n)})$.

2 Эквивалентность определений NP

Определение 2.1. Классом $\text{NTIME}(T(n))$ называется множество языков, распознаваемых на недетерминированной машине Тьюринга за время $O(T(n))$.

$$\text{NP} = \bigcup_{c=1}^{\infty} \text{NTIME}(n^c)$$

В названии класса $\mathbf{N}$ обозначает "недетерминированные вычисления", а $\mathbf{P}$ — "полиномиальное время".

Определение 2.2. Классом $\mathbf{NP}_\#$ называется множество языков $\mathcal{L}$, для которых существует функция $V(x, s)$ с булевыми значениями, вычисляемая за полиномиальное время от длины **первого аргумента** такая, что:

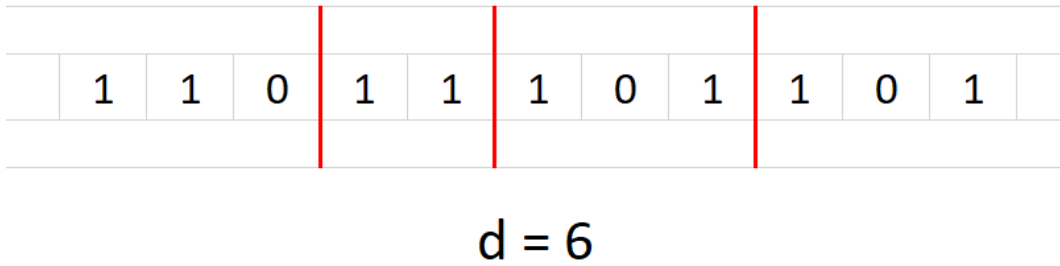
- Если $x \in \mathcal{L}$, то $\exists s V(x, s) = 1$
- Если $x \notin \mathcal{L}$, то $\forall s V(x, s) = 0$

Второй вход s часто называют **сертификатом**, а функцию V — **верификатором**. Таким образом, сертификат удостоверяет, что $x \in \mathcal{L}$, а верификатор проверяет верность сертификата.

Замечание 2.1. На самом деле мы сформулировали выше два определения $\mathbf{NP}$. Давайте докажем их эквивалентность.

Доказательство. ($\mathbf{NP} \subseteq \mathbf{NP}_\#$) Пусть $L \in \mathbf{NP} \implies$ существует полином $p(|x|)$ и НМТ, работающая $p(|x|)$ и принимающая L .

Построим ДМТ (на основе НМТ), принимающую в качестве s некоторую последовательность из 0 и 1 и "интерпретирующую" её как слово в алфавите $\{1, 2, \dots, d\}$, где d — максимальное количество вариантов в δ .



Каждый раз при эмуляции соответствующего шага НМТ будет выбирать вариант перехода, соответствующий последнему считанному числу (т.е. букве в алфавите $\{1, 2, \dots, d\}$). В случае отсутствия такого варианта достаточно обновить сертификат и пойти заново. Если при эмуляции получили q_a , то $V(x, s) = 1$, а иначе $V(x, s) = 0$. Тогда мы получили ДМТ, принимающую язык пар (x, s) , где $x \in L$, а s — подходящий сертификат. Это и говорит нам о том, что $L \in \mathbf{NP}_\# \implies \mathbf{NP} \subseteq \mathbf{NP}_\#$.

($\mathbf{NP}_\# \subseteq \mathbf{NP}$) Пусть $L \in \mathbf{NP}_\# \implies$ существуют ДМТ и $V(x, s)$.

Построим НМТ:

1. Недетерминированно генерируем на отдельной ленте слово в алфавите $\{0, 1\}$ (считаем, что $s \in \{0, 1\}^*$) длины не более $q(|x|)$. Тратим $O(q(|x|)) + O(\text{poly}(q(|x|)))$.
2. Эмулируем работу V , используя сгенерированное слово в качестве s . Тратим $O(\text{poly}(q(|x|)))$.

$\implies L \in \mathbf{NP} \implies \mathbf{NP}_\# \subseteq \mathbf{NP} \implies \mathbf{NP} = \mathbf{NP}_\#$ □

Пример 2.1. Рассмотрим язык $L = \{G \mid \text{существует правильная 3-раскраска } G\}$ из $\mathbf{NP}$. Тогда сертификатом как раз будет являться 3-раскраска графа, а верификатор будет проверять её корректность.

3 Сводимости по Карпу и Куку

Определение 3.1. Пусть A и B — два языка. Тогда A **сводится по Карпу** к B , если существует всюду определённая функция $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$, вычисляемая за полиномиальное время, такая что:

$$x \in A \iff f(x) \in B$$

Обозначение: $A \leq_p B$ (индекс p означает полиномиальность).

Лемма 3.1. Свойства сводимости по Карпу

1. $A \leq_p A$ (рефлексивность)
2. $A \leq_p B$ и $B \leq_p C \implies A \leq_p C$ (транзитивность)
3. $A \in P$, а $B \neq \emptyset$, $B \neq \{0, 1\}^* \implies A \leq_p B$
4. $A \leq_p B$ и $B \in P \implies A \in P$
5. $A \leq_p B$ и $B \in NP \implies A \in NP$

Замечание 3.1. Если сводимость по Карпу вам что-то напоминает, то вы правы. Она очень похожа на m -сводимость. Иногда её называют полиномиальной m -сводимостью.

Определение 3.2. Сведение задачи R_1 к R_2 **по Куку** — это полиномиальный по времени алгоритм (другими словами, МТ с полиномиальным временем работы), решающий задачу R_1 при условии, что функция, находящая решение задачи R_2 , ему дана в качестве оракула, то есть обращение к ней занимает всего один шаг.

Обозначение: $R_1 \leq R_2$.

Замечание 3.2. Определение приведено для ознакомления. В нашем курсе мы будем использовать именно сводимость по Карпу.

4 coNP

Определение 4.1. Для класса C класс coC определяется так: $L \in coC \iff \bar{L} \in C$.

Определение 4.2. Класс $coNP$ — это множество языков, дополнения которых лежат в классе NP .

Определение 4.3 (Сертификатное определение $coNP$). Классом $coNP$ называется множество языков L , для которых существует функция $V(x, s)$ с булевыми значениями, вычисляемая за полиномиальное время от длины **первого аргумента** такая, что:

- Если $x \in L$, то $\forall s V(x, s) = 1$
- Если $x \notin L$, то $\exists s V(x, s) = 0$

Лемма 4.1.

$$P \subseteq NP \cap coNP$$

Гипотеза 1.

$$NP \neq coNP$$

5 NP-трудность и NP-полнота

Определение 5.1. Язык B называется *NP-трудным*, если для любого $A \in \text{NP}$ выполнено $A \leq_p B$.

Определение 5.2. Язык B называется *NP-полным*, если он является NP-трудным и принадлежит классу NP.

Лемма 5.1. Если B является NP-трудным и $B \in \text{P}$, то $\text{P} = \text{NP}$.

Лемма 5.2. Если A является NP-трудным и $A \leq_p B$, то B также является NP-трудным.

6 Автор хоронит надежды понять формалки

